

Enterprise Cloud Architecture — Master Networking (Phases 01-05)

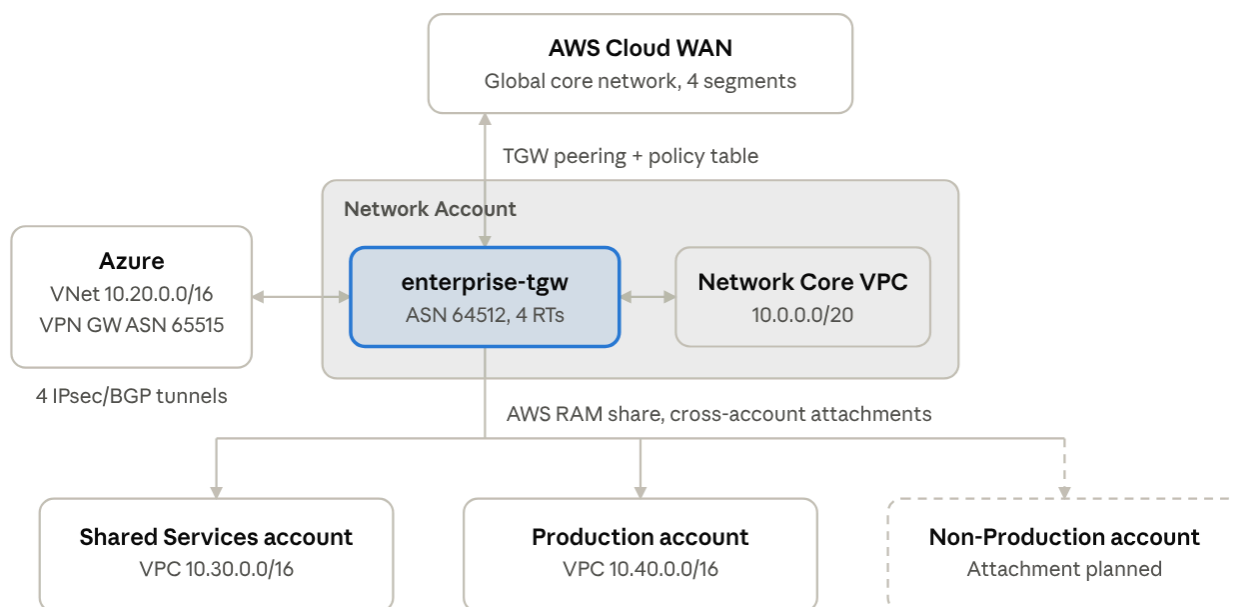
Oct 7, 2026 · @Dipesh Pawar

A hands-on, enterprise-grade AWS networking foundation built in five phases, from architecture design to multi-account, multi-cloud connectivity. Every phase follows the same workflow: Design → AWS Console → Validate → Terraform import → Reconcile drift → Document.

Full documentation, diagrams and Terraform: [PersonalProject / P12-NetworkingMasterArchitecture](#)

Architecture at a glance

One Transit Gateway in the Network Account connects everything



Dashed = planned. Prod and Non-Prod share no route.

End-to-end architecture after Phase 05

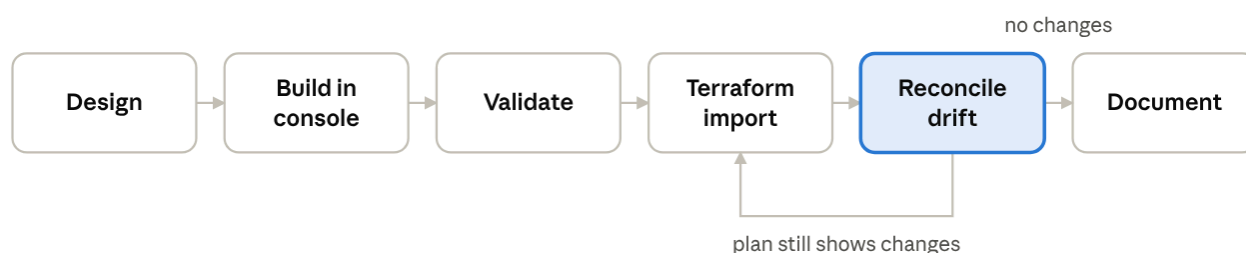
The Network Account owns the hub: one Transit Gateway with four route tables, peered to Cloud WAN for global reach and connected to Azure over four BGP tunnels. Workload accounts attach their VPCs to it through AWS RAM.

Phase	Focus	Outcome
01	Master architecture	AS-IS, gap analysis, TO-BE layers, ADRs

Phase	Focus	Outcome
02	Landing zone	3 OUs, 6 accounts, SCPs, Control Tower, Identity Center
03	Network Account core	IPAM, Network Core VPC, Transit Gateway, Cloud WAN
04	Hybrid connectivity	AWS ↔ Azure over 4 IPsec/BGP tunnels
05	Cross-account connectivity	RAM sharing, segmented TGW routing

How every phase was built

Every phase ends with a zero-change Terraform plan



Build workflow used in every phase

Resources are first built and understood in the console, then imported into Terraform. The phase is only done when terraform plan reports no changes, so the code describes what actually exists.

Phase 01 — Master Architecture

Defined the target enterprise architecture before creating any AWS resource: requirements first, services second.

- Business context and a fictional enterprise client migrating modern (EKS), legacy (EC2/NGINX) and data workloads to AWS
- AS-IS assessment → Gap Analysis → TO-BE design
- TO-BE layers: accounts, VPC and network, enterprise connectivity, edge, workload, middleware, data, security and observability
- Architecture Decision Records (ADRs) for every major choice

The client scenario

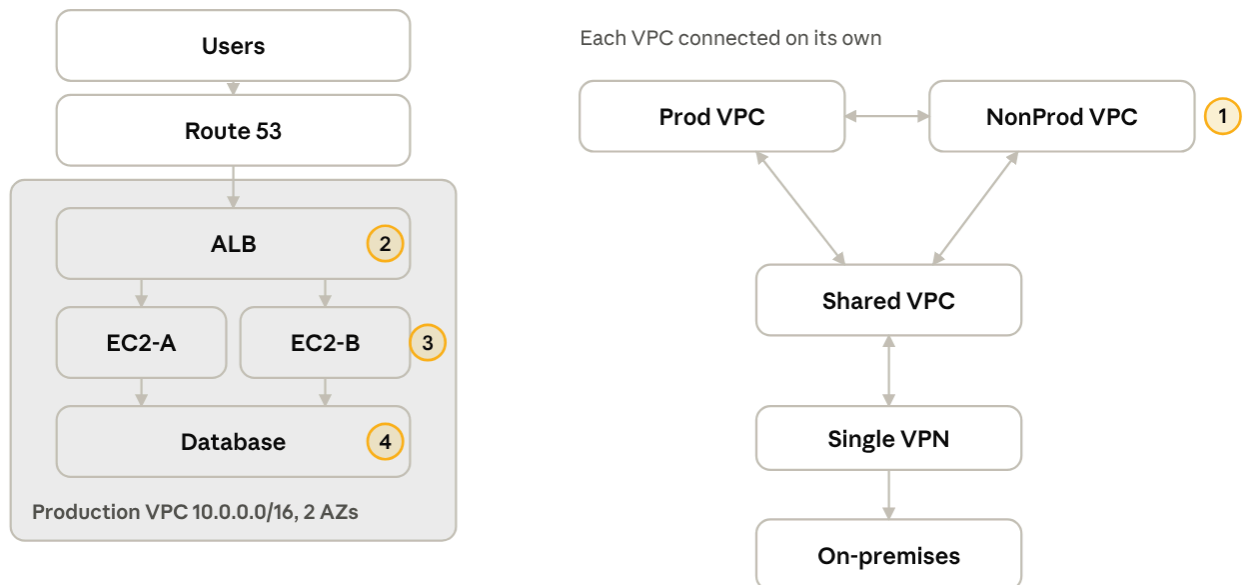
The exercise models a large enterprise migrating and modernizing its application platform onto AWS. It cannot move everything to Kubernetes at once, so the target must support modern and traditional workloads side by side.

Workload type	Runs on	Examples
Modern applications	Amazon EKS	Containerized microservices
Legacy applications	Amazon EC2	NGINX and Apache web tiers
Data layer	Managed data services	PostgreSQL, MySQL, MongoDB, Redis, Amazon MQ

AS-IS assessment

The current environment works, but it was never designed as one system. The assessment asked the questions an architect asks first: is it highly available, how do VPCs talk to each other, how is traffic inspected, and what happens when an AZ, a NAT Gateway or the VPN fails.

AS-IS works, but four gaps block an enterprise platform



- | | |
|--------------------------|---|
| 1 No central network hub | 2 No edge security layer (CloudFront, WAF) |
| 3 No container platform | 4 Every call hits the database: no cache or messaging |

AS-IS environment and its four gaps

Area	What exists today	Observation
Accounts	Production, Non-Production, Shared	No central security or log archive account
Production VPC	10.0.0.0/16 across 2 AZs, public and private subnets	Multi-AZ exists, HA still to be proven
Internet edge	Route 53 → ALB → EC2	No CloudFront, WAF or central firewall
VPC connectivity	Each VPC connected independently	Point-to-point mesh grows with every VPC
External connectivity	Single VPN to on-premises	No central connectivity model
Applications	EC2 behind an ALB, single database	No container platform
Security	IAM, Security Groups, NACLs	No central WAF, firewall or traffic inspection
Observability	Basic CloudWatch on EC2	Little cross-account or network visibility

Gap analysis

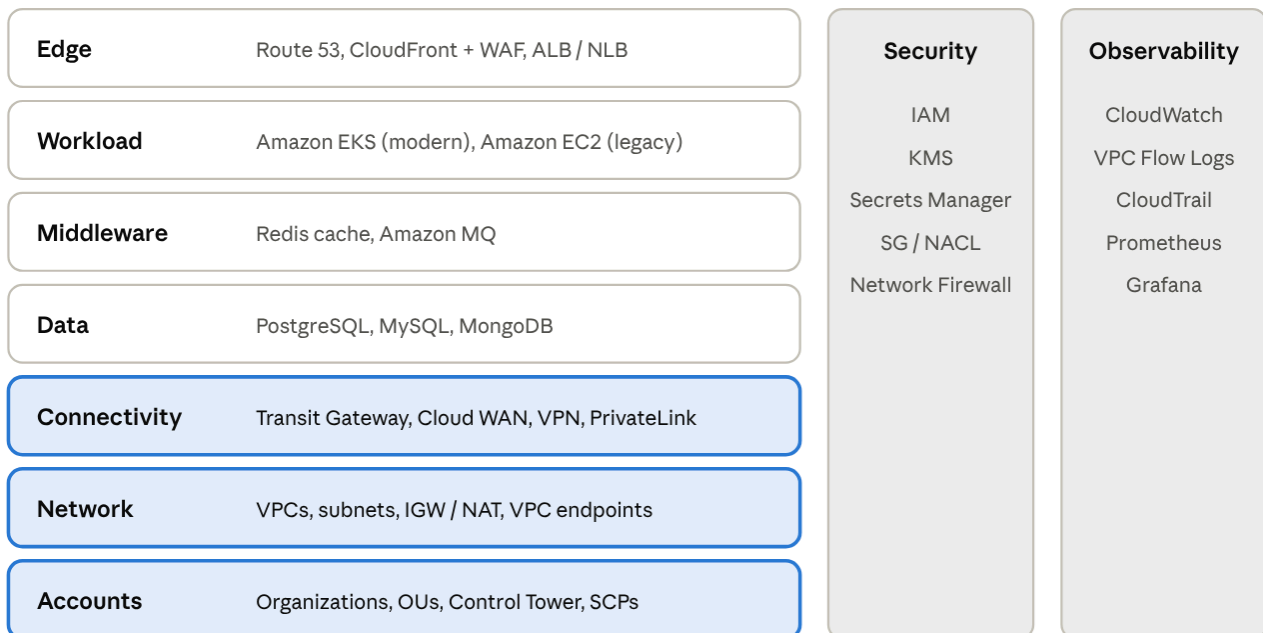
The design never jumps straight from AS-IS to TO-BE. Each gap is stated as current state, requirement and gap, and only then is a service chosen.

Gap	Requirement	What is missing	TO-BE answer
1. Network connectivity	Central, scalable connectivity	No network hub	Transit Gateway, later Cloud WAN
2. Edge security	Protect public application traffic	No complete edge security layer	Route 53 → CloudFront + WAF → ALB
3. Modernization	Containerized, scalable workloads	No orchestration platform	Amazon EKS alongside EC2
4. Database load	Fewer synchronous DB calls, async work	No caching or messaging layer	Redis and Amazon MQ

TO-BE architecture and key decisions

The TO-BE design is organized in layers, from accounts at the bottom to observability across everything. Security is not a hop in the traffic path: IAM, KMS, Secrets Manager, Security Groups, NACLs and Network Firewall apply across every layer.

Security and observability span every TO-BE layer



Highlighted layers are built in Phases 02 to 05; the rest come in later phases.

TO-BE architecture layers

ADR	Decision	Why	Trade-off
ADR-001	Deploy critical workloads across multiple AZs	Remove dependency on one AZ	Higher cost and complexity
ADR-002	Separate Production, Non-Production, Security/Log Archive and Shared Services accounts	Isolation, governance, smaller blast radius	More accounts to govern
ADR-003	Support both EKS and EC2	The enterprise runs modern and legacy apps	Two compute platforms to operate

TO-BE network design

Every workload VPC follows the same standard: a non-overlapping CIDR from IPAM, three subnet tiers per Availability Zone, and routes that send remote prefixes to the Transit Gateway.

Element	Standard
CIDR	One non-overlapping /16 or /20 per VPC, allocated from the regional IPAM pool
Public tier	ALB and NAT Gateway; 0.0.0.0/0 → Internet Gateway
Private application tier	EKS nodes and EC2; 0.0.0.0/0 → NAT Gateway in the same AZ
Private data tier	RDS, Redis, Amazon MQ; local routes plus TGW routes only where required
Transit Gateway routes	Remote VPC, Azure and on-premises prefixes → TGW
VPC endpoints	Gateway endpoints for S3/DynamoDB, interface endpoints for ECR, STS, CloudWatch Logs, Secrets Manager
Security Groups	Stateful, reference other Security Groups (ALB SG → App SG → DB SG)
NACLs	Stateless subnet guardrails that must allow ephemeral return ports

Enterprise traffic flows

The design was checked against the flows the platform must carry, not only against a list of services. Each flow names its path so it can be tested later.

Flow	Path
Internet → application	Route 53 → CloudFront (with WAF) → ALB → EKS or EC2
Application → cache	Application → Redis
Application → database	Application → PostgreSQL, MySQL or MongoDB in the data tier
Application → messaging	Producer → Amazon MQ → consumer
Workload → shared services	Workload VPC → TGW → Shared Services VPC
On-premises → AWS	Customer router → Site-to-Site VPN → TGW → VPC

Flow	Path
Azure → AWS	Azure VPN Gateway → IPsec/BGP → TGW → VPC
Private egress	Private subnet → NAT Gateway → Internet Gateway, inspected by Network Firewall in the target design

AWS WAF is not a separate network hop. A web ACL is attached to CloudFront or the ALB and evaluated inside that service; it cannot be attached to an NLB.

Phase 01 deliverables

- High-level and low-level architecture for every TO-BE layer
- AS-IS assessment and a gap register with a TO-BE answer for each gap
- Architecture Decision Records for multi-AZ, multi-account and EKS + EC2
- Traffic flows used as the test plan for later phases
- The phase roadmap: Landing Zone → Network Account → hybrid connectivity → cross-account connectivity → workloads

Phase 02 — AWS Landing Zone & Multi-Account

Built the governance foundation every later phase sits on, then imported all of it into Terraform until the plan showed no changes.

- AWS Organizations with 3 OUs: Security, Infrastructure, Workloads
- 6 member accounts: Security, Log Archive, Network, Shared Services, Production, Non-Production
- Service Control Policies: deny leaving the organization, protect CloudTrail
- AWS Control Tower landing zone with centralized logging and an AWS Config aggregator
- IAM Identity Center single sign-on across all 7 accounts

Why multiple accounts

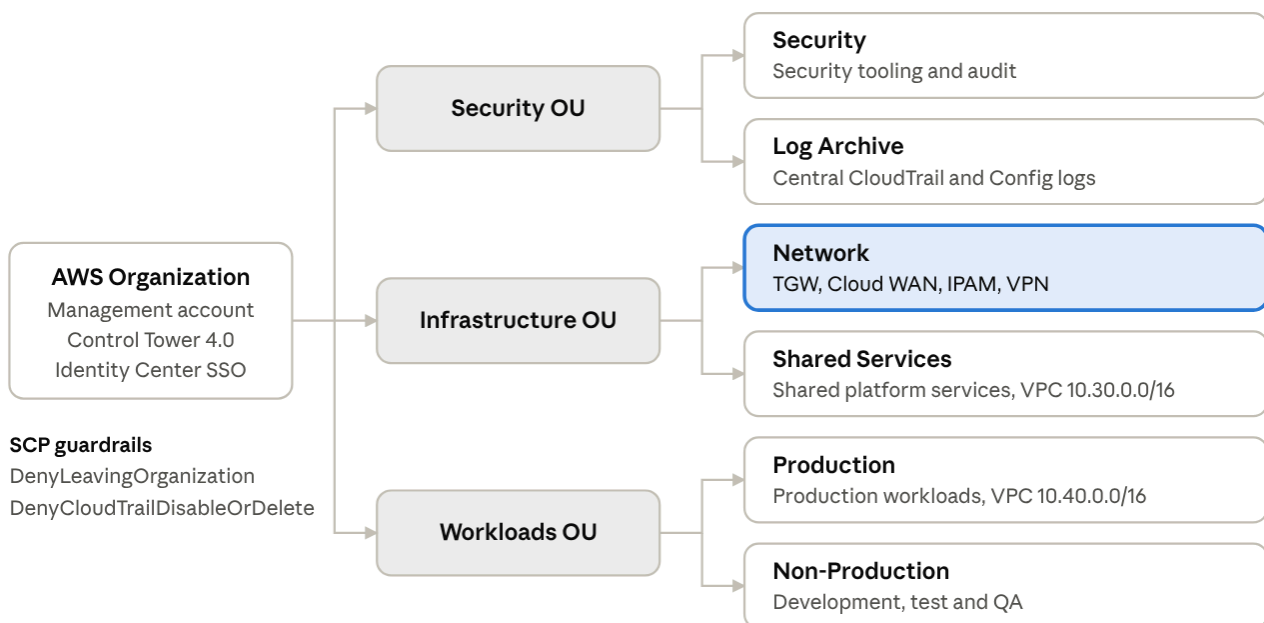
An AWS account is a hard security and billing boundary. Putting everything in one account weakens security boundaries, lets non-production affect production, lets workload admins touch logs, and makes the blast radius of any mistake the whole company. AWS Organizations does not carry any traffic; it is the governance layer the network sits on.

Organization structure

OU	Account	Responsibility
Security	Security	Security tooling and audit
Security	LogArchive	Central CloudTrail and Config logs
Infrastructure	Network	Transit Gateway, Cloud WAN, IPAM, VPN, central routing
Infrastructure	SharedServices	Shared platform services for all workloads
Workloads	Production	Production VPCs and applications
Workloads	NonProduction	Development, test and QA

The management account sits above the OUs and only runs Organizations, Control Tower, Identity Center and billing.

Six accounts in three OUs, each with one job



AWS Organization: OUs and member accounts

Guardrails and landing zone

Control	What it does
DenyLeavingOrganization (SCP)	Member accounts cannot leave the organization

Control	What it does
DenyCloudTrailDisableOrDelete (SCP)	Blocks cloudtrail:StopLogging and cloudtrail>DeleteTrail in member accounts
Control Tower Landing Zone 4.0	Governed Regions us-east-1 and us-east-2, AWS-managed controls
Centralized logging	CloudTrail and Config delivered to the LogArchive account
AWS Config aggregator	One compliance view across all accounts

An SCP never grants a permission. It only sets the maximum boundary, so IAM must still allow an action and the SCP must not deny it.

Identity

IAM Identity Center replaces long-lived IAM users. One Identity Center user signs in once and reaches all 7 accounts through an AdministratorAccess permission set with a 4-hour session. In a real enterprise this permission set would be split into read-only, network-admin and break-glass roles.

Terraform import

Everything was built in the console first, then imported into a Terraform organization module and reconciled until terraform plan reported no changes.

Imported into Terraform	Notes
Organization, 3 OUs, 6 accounts	OU names normalized ("Security OU" → "Security")
2 SCPs and their attachments	Attachment import ID format is TARGET_ID:POLICY_ID
Identity Center user, permission set, assignments	Permission set description limited to 700 characters

Implementation steps

1. Verified the management account and enabled AWS Organizations with all features.
2. Created the three OUs first, then created the six member accounts and moved each one into its OU.

3. Created the two SCPs and attached them to the OUs, not to the root, so the management account stays unaffected.
4. Enabled AWS Control Tower with the option for an existing environment, then registered each OU so its accounts were enrolled.
5. Enabled IAM Identity Center, created the admin user, registered MFA and created the AdministratorAccess permission set.
6. Assigned the permission set to the user in all seven accounts and confirmed sign-in through the access portal.
7. Wrote the Terraform organization module, imported every resource and reconciled drift until terraform plan reported no changes.

How it was validated

Check	Expected result
A member account tries to leave the organization	Denied by DenyLeavingOrganization
A member account tries to stop CloudTrail	Denied by DenyCloudTrailDisableOrDelete
Control Tower organization view	Root, 3 OUs and 7 accounts visible, management account enabled
Identity Center access portal	One sign-in reaches every account
terraform plan after import	No changes

What Phase 02 enables

The Network Account now exists as a separate, governed account. Every later phase builds inside it or attaches to it, and every resource it owns is already inside the guardrails.

Phase 03 — Network Account: Cloud WAN + Transit Gateway Initial Set-up

Built the central networking hub in the dedicated Network Account, with IP planning done before the first VPC.

- VPC IPAM: enterprise pool 10.0.0.0/8 with a regional us-east-1 pool, so no CIDRs overlap

- Network Core VPC 10.0.0.0/20: 9 subnets across 3 AZs (public, firewall, private), IGW, NAT and route tables
- Transit Gateway with 4 dedicated route tables (Network, Shared Services, Production, Non-Production) and default association/propagation disabled
- AWS Cloud WAN global network and core network, peered with the TGW, with one segment per environment
- IPAM, VPC, subnets, gateways and routes imported into Terraform

Build order

CIDR/IPAM → Core Network VPC → subnets and routes → IGW/NAT → Transit Gateway → Cloud WAN → hybrid VPN → Network Firewall → PrivateLink/endpoints → DNS → workload connectivity → Terraform reconciliation. Every step followed Console → Verify → Terraform import → Plan → Apply → zero drift.

IP address management first

Transit Gateway cannot fix overlapping CIDRs: a packet for 10.10.x.x cannot tell which of two VPCs owns it. IPAM therefore came before the first VPC, owned by the Network Account.

Pool	CIDR	Purpose
enterprise-private-pool	10.0.0.0/8	Enterprise-wide private address space
enterprise-us-east-1-pool	10.0.0.0/16 (+ 10.1.0.0/20)	Regional pool for us-east-1 VPCs
Allocation: network-core-vpc	10.0.0.0/20	4,096 addresses for the Network Core VPC

Design note: AWS IPAM does not see Azure or on-premises ranges. Azure uses 10.20.0.0/16, which sits inside the AWS 10.0.0.0/8 pool, so that range should be reserved in IPAM to stop a future VPC from overlapping it.

Network Core VPC

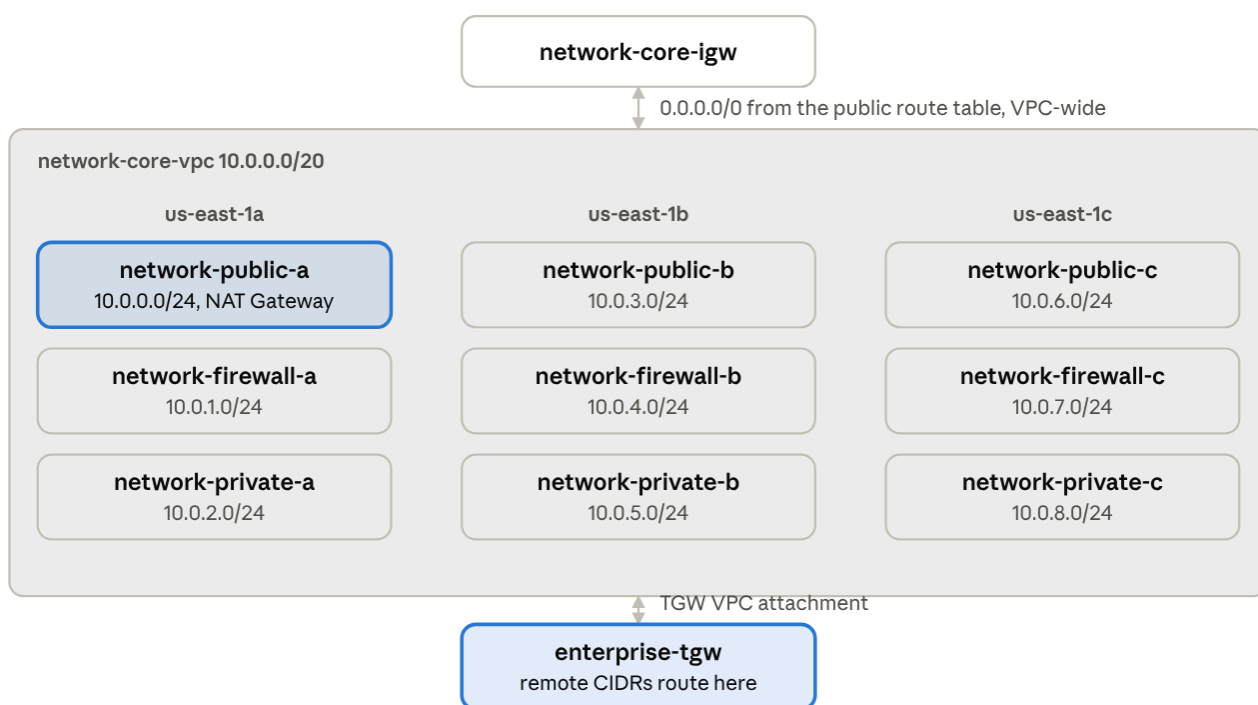
network-core-vpc (10.0.0.0/20) has three subnet tiers in each of three Availability Zones.

Tier	us-east-1a	us-east-1b	us-east-1c	Route
Public	10.0.0.0/24	10.0.3.0/24	10.0.6.0/24	0.0.0.0/0 → network-core-igw

Tier	us-east-1a	us-east-1b	us-east-1c	Route
Firewall	10.0.1.0/24	10.0.4.0/24	10.0.7.0/24	Reserved for Network Firewall endpoints
Private	10.0.2.0/24	10.0.5.0/24	10.0.8.0/24	0.0.0.0/0 → NAT, remote CIDRs → TGW

The design calls for one NAT Gateway per AZ. To control lab cost only one NAT Gateway was kept, which is a single-AZ dependency that a production build would not accept.

Network Core VPC: three tiers in each of three AZs



Firewall tier is reserved for Network Firewall endpoints. One NAT Gateway kept for lab cost.

network-core-vpc subnet layout

Transit Gateway

enterprise-tgw is the regional hub (ASN 64512). Default route-table association and propagation are disabled, so no attachment ever lands in a route table by accident.

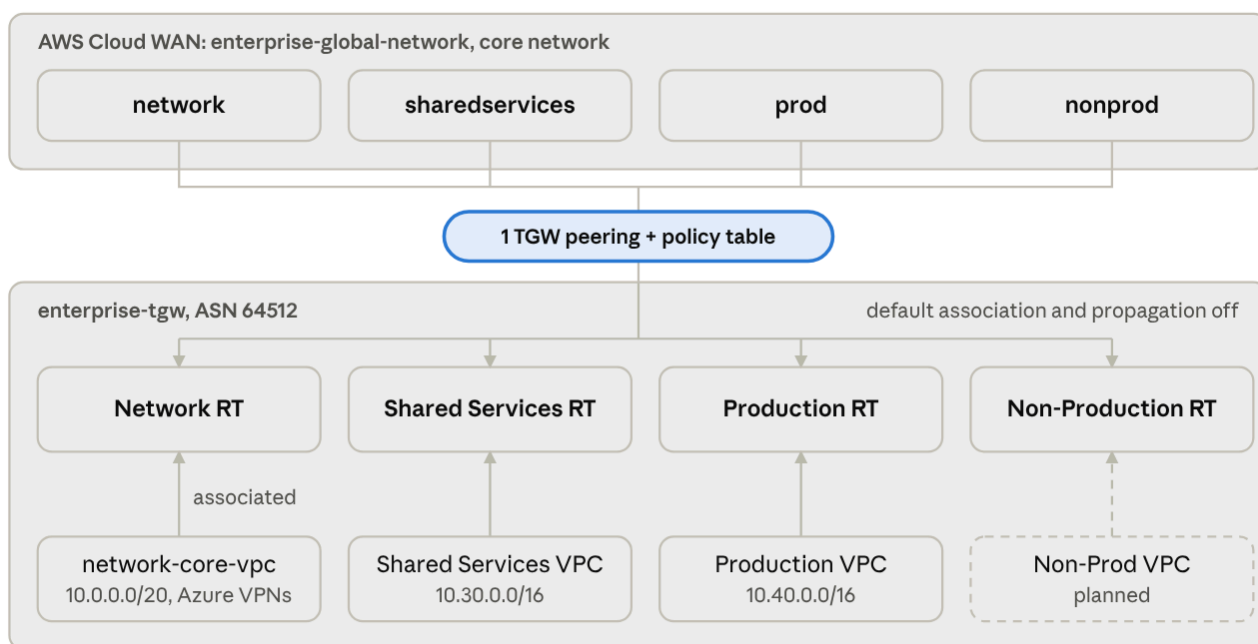
TGW route table	Associated attachment	Learns routes from
enterprise-tgw-rt-network	network-core-vpc, Azure VPNs	Network VPC, Shared Services, VPNs
enterprise-tgw-rt-shared-services	Shared Services VPC	Network VPC, Production

TGW route table	Associated attachment	Learns routes from
enterprise-tgw-rt-production	Production VPC	Shared Services
enterprise-tgw-rt-non-production	Non-Production VPC	Planned

Cloud WAN

AWS Cloud WAN provides the global layer: enterprise-global-network with one core network, peered to the TGW through a single peering, enterprise-cloudwan-tgw-peering. The peering carries a Transit Gateway policy table, which belongs to the peering itself and is not one of the four TGW route tables. Each TGW route table then maps to its Cloud WAN segment through a route-table attachment, so VPCs attach to the regional TGW and never directly to Cloud WAN.

One peering links four TGW route tables to four Cloud WAN segments



VPCs attach to the regional TGW, never directly to Cloud WAN.

Transit Gateway route tables and Cloud WAN segments

Terraform import

IPAM, both pools and their CIDRs, the VPC, 9 subnets, IGW, NAT Gateway, Elastic IP, route tables, routes and route-table associations were imported and applied to align tags. Route-table association imports only succeeded with the subnet-id/route-table-id ID format.

Implementation steps

1. Upgraded VPC IPAM to the Advanced tier and created the top-level enterprise pool with 10.0.0.0/8.
2. Created the regional us-east-1 pool from it and provisioned 10.0.0.0/16, plus 10.1.0.0/20 as a secondary range.
3. Created network-core-vpc with a /20 allocated from the regional pool.
4. Created the nine subnets across us-east-1a, 1b and 1c and associated each with its route table.
5. Created and attached the Internet Gateway and added 0.0.0.0/0 → IGW to the public route table.
6. Allocated three Elastic IPs and created NAT Gateways, then kept one NAT Gateway to save cost and pointed the private route table at it.
7. Created enterprise-tgw with ASN 64512 and default association and propagation turned off.
8. Created the four TGW route tables, attached the Network VPC, associated the attachment with the Network route table and propagated its CIDR.
9. Created the Cloud WAN global network and core network with segments for network, sharedservices, prod and nonprod.
10. Peered the core network with the TGW (enterprise-cloudwan-tgw-peering) and attached the TGW route tables to their segments.
11. Imported everything into Terraform and applied only tag alignment.

Design decisions

Decision	Why	Trade-off
IPAM before the first VPC	Overlapping CIDRs cannot be fixed by routing later	Advanced tier has a cost
Default TGW association and propagation off	No attachment can reach anything by accident	Every attachment needs explicit association
One TGW route table per environment	Routing decides who can talk to whom	More route tables to manage
Dedicated firewall subnet tier	Room for Network Firewall endpoints without re-addressing	Subnets sit unused until inspection is added
Cloud WAN peered to TGW, VPCs on the TGW	Regional hub stays simple, global layer is added once	Two services to understand

Decision	Why	Trade-off
One NAT Gateway in the lab	Lower cost while learning	Single-AZ dependency for egress

Errors met and fixed

- Route table association imports failed until the ID format subnet-id/route-table-id was used.
- The route import needed the route-table-id_destination-CIDR format, for example `rtb-..._0.0.0.0/0`.
- A Cloud WAN policy table belongs to the TGW peering and is a different object from the four TGW route tables.

Phase 04 — AWS ↔ Azure Hybrid Connectivity

Connected Azure to the AWS Transit Gateway over a redundant, BGP-driven site-to-site VPN, proven with an EC2 → Azure VM ping (4/4 packets).

- Azure: VNet 10.20.0.0/16, GatewaySubnet, active-active VPN Gateway (VpnGw2AZ, ASN 65515) with 2 public IPs and APIPA BGP addresses
- AWS: 2 Customer Gateways and 2 Site-to-Site VPN connections terminating on the TGW (ASN 64512)
- 4 IPsec tunnels with BGP, plus 4 Azure Local Network Gateways and IKEv2 connections
- Routing: VPN attachments propagated into the TGW route table, VPC route 10.20.0.0/16 → TGW, and 10.0.0.0/20 advertised to Azure by BGP
- Both clouds imported into Terraform (AWS and AzureRM providers)

Azure side

Resource	Name	Setting
Resource group	rg-hybrid-networking	Central India
VNet	vnet-hybrid-azure	10.20.0.0/16
Workload subnet	snet-workload	10.20.1.0/24
Gateway subnet	GatewaySubnet	10.20.255.0/24
VPN gateway	vng-hybrid-azure	VpnGw2AZ, route-based, active-active, BGP, ASN 65515

Resource	Name	Setting
Public IPs	pip-vpngw-azure, pip-vpngw-azure-02	One per gateway instance
APIPA BGP addresses	Instance 0 / Instance 1	169.254.21.2, 169.254.22.2 / 169.254.21.6, 169.254.22.6
Test VM	vm-azure-connectivity-test	10.20.1.4, no public IP

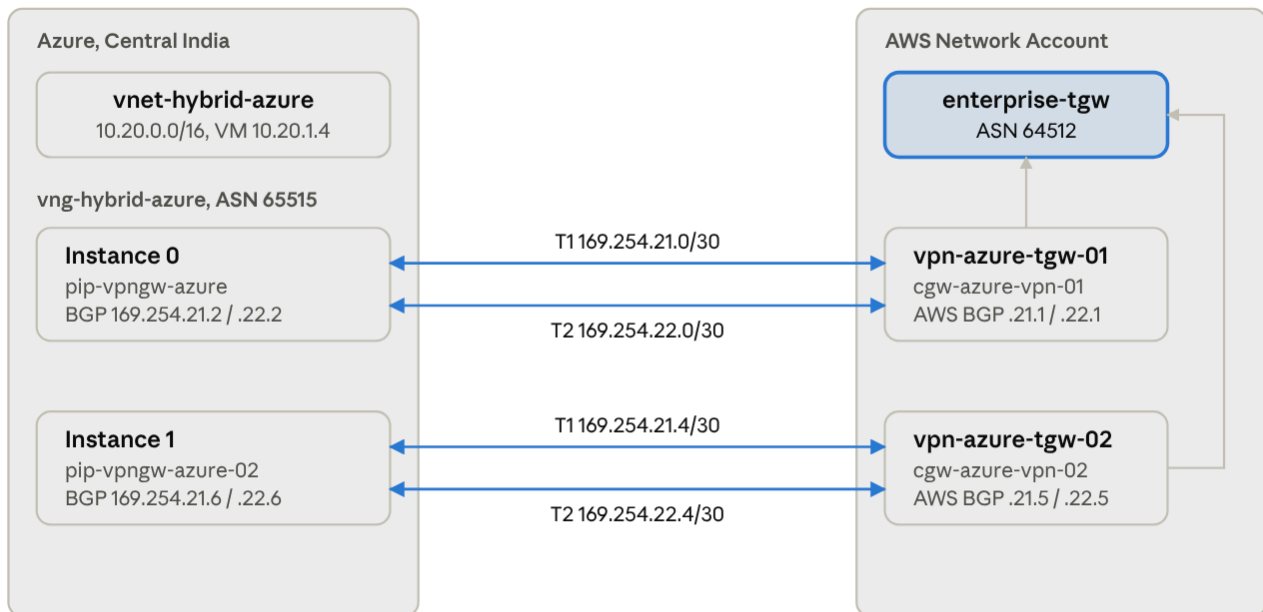
AWS side

The Customer Gateway does not create tunnels. It only represents the Azure endpoint inside AWS: cgw-azure-vpn-01 and cgw-azure-vpn-02, one per Azure public IP, both with BGP ASN 65515. Each Site-to-Site VPN connection (vpn-azure-tgw-01 and vpn-azure-tgw-02) brings two tunnels and terminates on the Transit Gateway, so two Azure IPs give four tunnels.

Tunnel	AWS public endpoint	Inside CIDR	AWS BGP	Azure BGP
VPN-01 T1	34.239.234.228	169.254.21.0/30	169.254.21.1	169.254.21.2
VPN-01 T2	100.27.76.0	169.254.22.0/30	169.254.22.1	169.254.22.2
VPN-02 T1	3.216.209.232	169.254.21.4/30	169.254.21.5	169.254.21.6
VPN-02 T2	34.232.29.55	169.254.22.4/30	169.254.22.5	169.254.22.6

Back in Azure, each AWS tunnel became one Local Network Gateway (lng-aws-tgw-01-t1 ... 02-t2) holding the tunnel's public IP, ASN 64512 and BGP peer address, and one IKEv2 connection (conn-aws-tgw-01-t1 ... 02-t2) authenticated with a pre-shared key.

Two Azure gateway instances, four BGP tunnels to the TGW



Public IPs build the IPsec tunnels; BGP runs over the 169.254.x.x inside addresses.
BGP advertises 10.0.0.0/20 to Azure and 10.20.0.0/16 to the TGW.

AWS ↔ Azure VPN topology, 4 tunnels

Routing

1. The two VPN attachments are associated with enterprise-tgw-rt-network and propagate the Azure prefix into it.
2. The Network VPC private route table sends 10.20.0.0/16 to the Transit Gateway.
3. BGP advertises 10.0.0.0/20 to Azure, so the Azure gateway learns the return path without static routes.

The public IPs build the IPsec tunnels, while the APIPA addresses carry the BGP sessions inside them. BGP is the control plane and IPsec plus TGW routing is the data plane: a tunnel can be UP while traffic still fails if a route table entry is missing.

Packet flow, AWS EC2 → Azure VM

Validation

ICMP from AWS EC2 10.0.2.236 to Azure VM 10.20.1.4 returned 4 of 4 packets, proving the data plane end to end.

Terraform import

Cloud	Imported	Drift reconciled
AWS	2 Customer Gateways, 2 VPN connections	Tags only (4 changes)
Azure	RG, VNet, 2 subnets, 2 public IPs, VPN gateway, 4 LNGs, 4 connections	Tags, IP-config names, custom BGP addresses (13 changes)

The Azure plan also flagged the deprecated `enable_bgp` argument, which AzureRM v5.0 replaces with `bgp_enabled`.

Build sequence

The two clouds were built in a fixed order, because each side needs values the other side only produces after it exists.

1. Azure: resource group, VNet, workload subnet and GatewaySubnet.
2. Azure: two public IPs and the active-active VPN Gateway with BGP and custom APIPA addresses.
3. AWS: two Customer Gateways from the Azure public IPs, with BGP ASN 65515.
4. AWS: two Site-to-Site VPN connections on the Transit Gateway, with dynamic routing and the tunnel inside CIDRs set to match Azure.
5. AWS: read the four tunnel public endpoints and BGP addresses.
6. Azure: four Local Network Gateways, one per AWS tunnel, then four IKEv2 connections with the pre-shared keys.
7. AWS: associated and propagated the VPN attachments in the TGW route table, then added the 10.20.0.0/16 route to the VPC route table.
8. Both: created a test VM and NSG in Azure, then pinged it from EC2.
9. Both: imported every resource into Terraform and reconciled drift.

ASN and address plan

Item	Value
Azure VPN Gateway ASN	65515
AWS Transit Gateway ASN	64512
Tunnel inside CIDRs	169.254.21.0/30, 169.254.22.0/30, 169.254.21.4/30, 169.254.22.4/30

Item	Value
Azure prefix advertised to AWS	10.20.0.0/16
AWS prefix advertised to Azure	10.0.0.0/20

Troubleshooting checklist

Symptom	Check	Typical cause
Tunnel stays DOWN	Public IPs, PSK and IKE version on both sides	PSK or IKEv2 mismatch
Tunnel UP, BGP down	Inside CIDR and APIPA peer addresses	Azure custom BGP address not matching the AWS inside IP
BGP up, no traffic	TGW association and propagation	VPN attachment not propagated into the route table
Traffic one way only	VPC route table and BGP routes learned in Azure	Missing return route or prefix not advertised
Ping fails, routes correct	Security Group, NACL and Azure NSG	ICMP not allowed

A tunnel being UP only proves IKE and IPsec. It says nothing about routing or filtering, which is why the checklist walks the packet path in order.

Security notes

- The pre-shared keys are secrets. They stay out of diagrams and documents and belong in a secret store such as Secrets Manager or Key Vault.
- The Azure test VM has no public IP and is reached only through the tunnels.

Phase 05 — Cross-Account Connectivity & Segmentation

Turned the central Transit Gateway into a multi-account hub with deliberate, segmented routing.

- Enterprise TGW shared from the Network Account via AWS RAM with the Shared Services, Production and Non-Production accounts
- Cross-account VPC attachments: Shared Services 10.30.0.0/16 and Production 10.40.0.0/16

- Each attachment associated with its own TGW route table, and propagation only where traffic should flow (Prod ↔ Shared, Network ↔ Shared; Prod ↔ NonProd isolated)
- VPC route tables pointing remote CIDRs to the TGW in both directions
- TGW route tables mapped to Cloud WAN segments (network, sharedservices, prod, nonprod) for multi-region growth

Sharing the Transit Gateway with AWS RAM

Workload accounts never get their own TGW. The Network Account shares enterprise-tgw through the RAM resource share enterprise-tgw-cross-account-share with the Shared Services, Production and Non-Production accounts. Each account then creates its own VPC attachment to the shared TGW.

Association versus propagation

This is the most important idea in the phase. Association decides which TGW route table an attachment uses to look up its traffic. Propagation decides which route tables learn that attachment's CIDR.

Attachment	Associated with	Propagates into
Network Core VPC (10.0.0.0/20)	Network RT	Network RT, Shared Services RT
Shared Services VPC (10.30.0.0/16)	Shared Services RT	Network RT, Production RT
Production VPC (10.40.0.0/16)	Production RT	Shared Services RT
Non-Production VPC	Non-Production RT	Planned

Because Production never propagates into a Non-Production route table, and the reverse, the two environments have no route to each other. Segmentation is a routing decision, not only a firewall rule.

TGW association and propagation, Phase 05

VPC route tables

TGW propagation never touches VPC route tables, so each VPC needed an explicit route to the remote CIDR via the TGW, in both directions.

VPC route table	Destination	Target
enterprise-shared-private-rt (Shared Services)	10.0.0.0/20	enterprise-tgw
enterprise-production-private-rt (Production)	10.30.0.0/16	enterprise-tgw
Network private route table	10.20.0.0/16 (Azure)	enterprise-tgw

A route existing does not mean traffic is allowed. A packet must pass the source VPC route table, the TGW route table, the destination VPC route table, the Security Group, the NACL and finally the application.

Routing matrix

Source	Destination	Path	Expected
Production	Shared Services	TGW	Allowed
Shared Services	Production	TGW	Allowed
Network	Shared Services	TGW	Allowed
Production	Non-Production	No route	Blocked
Region 1	Region 2	Cloud WAN	Designed, not deployed

Cloud WAN segments and HA

Each TGW route table maps to one Cloud WAN segment: network, sharedservices, prod and nonprod. TGW stays the regional hub and Cloud WAN carries traffic between Regions, so a second Region such as us-west-2 joins by peering its own TGW. The failure cases to test next are an AZ failure, a TGW attachment failure, a VPN path failure, a Cloud WAN path failure and a Region failure.

Multi-region target with Cloud WAN

Terraform

The Production account's VPC, two private subnets (10.40.1.0/24 and 10.40.2.0/24), route table, TGW route and TGW attachment are managed as code. The only drift was `enable_dns_hostnames`, switched from false to true.

Recommendation: the RAM share currently allows sharing with anyone. Since every account sits in one Organization, set it to share only within the organization.

Implementation order

1. Network Account: verified enterprise-tgw and its four route tables.
2. Network Account: created the RAM resource share for the TGW and added the three workload accounts as principals.
3. Workload accounts: accepted the share where needed, created each VPC and its private subnets.
4. Workload accounts: created the TGW VPC attachment from their own account to the shared TGW.
5. Network Account: associated each new attachment with its environment's TGW route table.
6. Network Account: configured propagation only for the flows in the routing matrix.
7. Workload accounts: added VPC routes to the remote CIDRs through the TGW, in both directions.
8. Verified the TGW routes, then the Security Groups and NACLs, then tested traffic.

Security validation layers

Connectivity was checked layer by layer, because each layer can block a packet that the layer before allowed.

Layer	Question it answers
Source VPC route table	Does the source send this prefix to the TGW?
TGW route table (association)	Which table does this attachment look up?
TGW route table (propagation)	Has that table learned the destination?
Destination VPC route table	Is there a route back to the source?
Security Group	Does the destination allow this port from this source?
NACL	Do both subnets allow the traffic and its ephemeral return ports?
Application	Is the service listening and healthy?

Failure scenarios to test next

Scenario	Expected impact	Mitigation in the design
One Availability Zone fails	Traffic through that AZ's attachment ENI stops	TGW attachment subnets in at least two AZs
TGW attachment fails	The VPC loses cross-account reach	Monitor attachment state, recreate from Terraform
One VPN tunnel fails	Traffic shifts to the remaining tunnels	Four tunnels with BGP failover
Cloud WAN path fails	Inter-region traffic stops, regional traffic continues	Regional TGW keeps local routing independent
Region fails	Workloads in that Region are lost	Second Region joined through Cloud WAN
NAT Gateway AZ fails (lab)	Private subnets lose Internet egress	One NAT Gateway per AZ in production

Cost control in the lab

The project runs on a personal budget, so every component was built, tested, documented and then deleted or scaled down. The lab keeps production-grade design while trading away some resilience.

Component	Cost driver	How it was controlled
NAT Gateway	Hourly charge per gateway plus data processing	One NAT Gateway kept instead of one per AZ
Transit Gateway	Hourly charge per attachment plus data processing	Only the attachments a test needs
Site-to-Site VPN	Hourly charge per VPN connection	Torn down after validation, recreated from Terraform
Azure VPN Gateway	Hourly charge by SKU	Deleted after testing
Cloud WAN	Core network edge and attachment hours	Single Region, design for multi-region only
IPAM Advanced tier	Charge per active IP address managed	Small address footprint

Component	Cost driver	How it was controlled
Test VMs	Compute hours	Smallest sizes, no public IPs, stopped after tests

Because every resource is in Terraform, deleting the expensive parts carries no risk: terraform apply rebuilds them in minutes.

Tools and skills used

Area	Tools
Governance	AWS Organizations, SCPs, AWS Control Tower, AWS Config, CloudTrail
Identity	IAM Identity Center, permission sets, MFA
Networking	VPC IPAM, VPC, subnets, route tables, IGW, NAT Gateway, Transit Gateway, AWS Cloud WAN, AWS RAM
Hybrid	AWS Site-to-Site VPN, Customer Gateway, Azure VPN Gateway, Local Network Gateway, IPsec, IKEv2, BGP
Infrastructure as code	Terraform (AWS and AzureRM providers), terraform import, drift reconciliation
CLI	AWS CLI, Azure CLI
Documentation	Architecture diagrams, ADRs, routing matrices, interview notes

Interview-ready explanations

Short answers to the questions this project is built to handle, in the words I would use with an interviewer.

How did you design the account structure, and why?

I used AWS Organizations with three OUs: Security for the Security and Log Archive accounts, Infrastructure for the Network and Shared Services accounts, and Workloads for Production and Non-Production. Each account is a hard boundary, so a mistake in Non-

Production cannot touch Production, and workload admins cannot change the logs. SCPs set the outer limit of what any account can do, Control Tower applies managed controls in the governed Regions, and Identity Center gives one sign-in across all accounts.

Why did you plan IPAM before creating any VPC?

Transit Gateway, VPN and Cloud WAN all route by destination prefix, so two networks with the same CIDR cannot be told apart. I created an enterprise 10.0.0.0/8 pool and a regional us-east-1 pool, then allocated the Network Core VPC from it. Ranges outside AWS, such as Azure's 10.20.0.0/16, must be tracked in the same plan because IPAM cannot see them.

What is the difference between TGW association and propagation?

Association decides which TGW route table an attachment uses to look up where its traffic goes. Propagation decides which route tables learn that attachment's CIDR. Production is associated with the Production route table, which only learns Shared Services, so Production can reach Shared Services and nothing else.

Why use both Transit Gateway and Cloud WAN?

Transit Gateway is a regional hub. Cloud WAN is a managed global network with segments and policies. VPCs attach to the regional TGW, and the TGW peers with Cloud WAN, with each TGW route table mapped to a segment. Adding a Region means adding a TGW and one peering, not rebuilding every connection.

How did you connect Azure to AWS?

I built an active-active Azure VPN Gateway with ASN 65515 and two public IPs. In AWS, each public IP became a Customer Gateway, and each Customer Gateway got a Site-to-Site VPN connection terminating on the TGW, which gives four tunnels. BGP runs over APIPA inside addresses, the TGW route table learns 10.20.0.0/16, the VPC route table sends that prefix to the TGW, and BGP advertises 10.0.0.0/20 back to Azure. I proved it with a 4/4 ping from EC2 to an Azure VM.

A tunnel is UP but traffic fails. Where do you look?

I follow the packet: the source VPC route table, the TGW route table and its association, propagation of the destination prefix, the BGP routes learned on both sides, the destination route table, then the Security Group and NACL. A tunnel being UP only proves IKE and IPsec, not routing or filtering.

How do you keep Production and Non-Production isolated on a shared TGW?

Each environment has its own TGW route table, and neither propagates into the other. With no route, no packet can cross, whatever the Security Groups say. Firewall rules then add defense in depth on top of that routing isolation.

Why build in the console first and then import into Terraform?

The console shows exactly what AWS creates and which defaults it applies. Importing those resources into Terraform then exposes every difference between what I clicked and what I coded. I reconcile the drift until terraform plan shows no changes, so the code describes reality instead of an assumption.

Key lessons

1. Plan the IP space before the first VPC. Overlapping CIDRs cannot be routed around.
2. Association picks the TGW route table, and propagation fills it. Segmentation is a routing decision.
3. Public IPs build the IPsec tunnel, while APIPA addresses carry BGP inside it.
4. A route existing does not mean traffic is allowed: VPC RT → TGW RT → Security Group → NACL → application.
5. The first terraform plan after an import shows the real drift between console and code.

Next: On-Prem ↔ AWS connectivity, Network Firewall with centralized inspection, PrivateLink, hybrid DNS, then EKS networking.